

Programming Language Principles, Design, and Implementation

Compilers - Semantic Analysis

Vincent Rahli

University of Birmingham

Where are we?

- ▶ Part 1: Principles of programming
- ▶ **Part 2: Compilers**

Today

Semantic Analysis

- ▶ Semantic actions
- ▶ Abstract syntax
- ▶ Symbol tables

Further reading:

- ▶ Chapter 4 of “Modern Compiler Implementation”
- ▶ Section 2.7 of the Dragon book

Problem

- ▶ **Lexer** detects illegal tokens
- ▶ **Parser** detects whether input is part of the language
- ▶ **Semantic analysis**:
 - ▶ checks the semantics of the program
 - ▶ e.g., variables are declared, expressions have the right types

Example

Good Haskell program:

```
1 module Main where
2 x :: Bool
3 x = True
4
5 main :: IO ()
6 main = print "done"
7   where z = if x then 0 else 1
```

Bad Haskell program:

```
1 module Main where
2 x :: Bool
3 x = True
4
5 main :: IO ()
6 main = print "done"
7   where z = if y then "0" else 1
```

Semantic Actions

The parsers we discussed so far do not do anything useful with the sentences. We can extend them with **semantic actions**.

Consider this grammar:

$$\begin{aligned} E &::= E + T \mid T \\ T &::= T * F \mid F \\ F &::= (E) \mid num \end{aligned}$$

We indicate semantic actions using curly braces:

$$\begin{aligned} E &::= E + T \quad \{\text{print}(' + ')\} \\ E &::= T \quad \{\} \\ T &::= T * F \quad \{\text{print}(' * ')\} \\ T &::= F \quad \{\} \\ F &::= (E) \quad \{\} \\ F &::= num \quad \{\text{print}(num)\} \end{aligned}$$

Note: calling *num* returns the value of the token *num* (i.e., a number).

Semantic Actions

What else can we do apart from printing symbols?

We can write an **interpreter**:

$$\begin{array}{lll} E & ::= & E + T \quad \{E() + T()\} \\ E & ::= & T \quad \{T()\} \\ T & ::= & T * F \quad \{T() * F()\} \\ T & ::= & F \quad \{F()\} \\ F & ::= & (E) \quad \{E()\} \\ F & ::= & num \quad \{num\} \end{array}$$

Note:

- ▶ The rules all return a number now
- ▶ $E()$ returns the value returned by the semantic action called when parsing E , and similarly for the other non-terminal symbols

Semantic Actions

The semantic values can be kept in a stack when parsing:

stack	input	action								
<table><tr><td></td></tr><tr><td>\$</td></tr></table>		\$	42 + 35\$	shift						
\$										
<table><tr><td></td><td></td></tr><tr><td>\$</td><td>42</td></tr></table>			\$	42	+35\$	reduce using $F ::= num$				
\$	42									
<table><tr><td></td><td>42</td></tr><tr><td>\$</td><td>$F$</td></tr></table>		42	\$	F	+35\$	reduce using $T ::= F$				
	42									
\$	F									
<table><tr><td></td><td>42</td></tr><tr><td>\$</td><td>$T$</td></tr></table>		42	\$	T	+35\$	reduce using $E ::= T$				
	42									
\$	T									
<table><tr><td></td><td>42</td></tr><tr><td>\$</td><td>$E$</td></tr></table>		42	\$	E	+35\$	shift				
	42									
\$	E									
<table><tr><td></td><td>42</td><td></td></tr><tr><td>\$</td><td>$E$</td><td>+</td></tr></table>		42		\$	E	+	35\$	shift		
	42									
\$	E	+								
<table><tr><td></td><td>42</td><td></td><td></td></tr><tr><td>\$</td><td>$E$</td><td>+</td><td>35</td></tr></table>		42			\$	E	+	35	\$	reduce using $F ::= num$
	42									
\$	E	+	35							
<table><tr><td></td><td>42</td><td></td><td>35</td></tr><tr><td>\$</td><td>$E$</td><td>+</td><td>$F$</td></tr></table>		42		35	\$	E	+	F	\$	reduce using $T ::= F$
	42		35							
\$	E	+	F							
<table><tr><td></td><td>42</td><td></td><td>35</td></tr><tr><td>\$</td><td>$E$</td><td>+</td><td>$T$</td></tr></table>		42		35	\$	E	+	T	\$	reduce using $E ::= E + T$
	42		35							
\$	E	+	T							
<table><tr><td></td><td>77</td></tr><tr><td>\$</td><td>$E$</td></tr></table>		77	\$	E	\$	accepted				
	77									
\$	E									

Semantic Actions

Another example:

S	$::= S; S$	$\{\backslash t \rightarrow S2(S1(t))\}$
S	$::= [S]$	$\{\backslash t \rightarrow \text{let } _ = S(t) \text{ in } t\}$
S	$::= id := E$	$\{\backslash t \rightarrow \text{update}(t, id, E(t))\}$
E	$::= E + T$	$\{\backslash t \rightarrow E(t) + T(t)\}$
E	$::= T$	$\{\backslash t \rightarrow T(t)\}$
T	$::= T * F$	$\{\backslash t \rightarrow T(t) * F(t)\}$
T	$::= F$	$\{\backslash t \rightarrow F(t)\}$
F	$::= (E)$	$\{\backslash t \rightarrow E(t)\}$
F	$::= num$	$\{\backslash t \rightarrow num\}$
F	$::= id$	$\{\backslash t \rightarrow t(id)\}$

Note:

- ▶ num returns a number, and id a string
- ▶ A lookup table maps strings to numbers
- ▶ S returns a lookup table, and E, T, F numbers
- ▶ We write $S1$ for the first occurrence of S in $S; S$, and $S2$ for its second occurrence

Semantic Actions

What do we obtain when parsing $x := 2; y := x + 1$?

The lookup table $\{x \mapsto 2, y \mapsto 3\}$

What do we obtain when parsing $[x := 2]; y := x + 1$?

looking up x fails

What do we obtain when parsing $[x := 2; y := x + 1]$?

The lookup table $\{\}$

Abstract Syntax Tree

We could write the **entire compiler** as **semantic actions**.

Issues:

- ▶ hard to read and maintain
- ▶ constrains the compiler to analyze the program in the order it is parsed

Better to **separate syntax** (parsing) and **semantics** (type-checking & translation to machine code).

Semantic actions can store the parse tree in a data structure

However, the **concrete parse tree is inconvenient to use**

- ▶ useless punctuation symbols
- ▶ too tied to the grammar, which might contain extra non-terminal symbols (why would we want to distinguish between E , T , and F after parsing?)

Instead, we generate an **Abstract Syntax Tree** (AST)

Abstract Syntax Tree

AST for the above grammar (in Haskell):

```
1 data Stmt =  
2   SEQ Stmt Stmt  
3   | BLOCK Stmt  
4   | ASSIGN String Exp  
5  
6 data Exp =  
7   PLUS Exp Exp  
8   | MULT Exp Exp  
9   | NUM Integer  
10  | ID String
```

S	$::=$	$S; S$	{SEQ (S1()) (S2())}
S	$::=$	$[S]$	{BLOCK (S())}
S	$::=$	$id := E$	{ASSIGN id (E())}
E	$::=$	$E + T$	{PLUS (E()) (T())}
E	$::=$	T	{T()}
T	$::=$	$T * F$	{MULT (T()) (F())}
T	$::=$	F	{F(t)}
F	$::=$	(E)	{E(t)}
F	$::=$	num	{NUM(num)}
F	$::=$	id	{ID(id)}

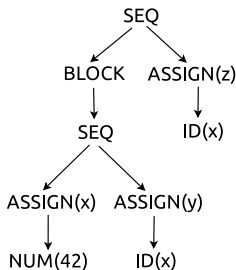
BNF:

$$\begin{aligned} S &::= S; S \mid [S] \mid id := E \\ E &::= E + E \mid E * E \mid num \mid id \end{aligned}$$

Abstract Syntax Tree

What is the AST of $[x := 42; y := x]; z := x$?

SEQ (BLOCK (SEQ (ASSIGN x (NUM 42)) (ASSIGN y (ID x)))) (ASSIGN z (ID(x)))



Abstract Syntax Tree

An interpreter can now be built separately (in Haskell):

```
1 type Env = String -> Integer
2 update e id exp = \id' -> if id == id' then exp else e id'
3
4 interpE :: Exp -> Env -> Integer
5 interpE (PLUS e1 e2) e = interpE e1 e + interpE e2 e
6 interpE (MULT e1 e2) e = interpE e1 e * interpE e2 e
7 interpE (NUM i) e = i
8 interpE (ID i) e = e i
9
10 interpS :: Stmt -> Env -> Env
11 interpS (SEQ s1 s2) e = interpS s2 (interpS s1 e)
12 interpS (BLOCK s) e = let _ = interpS s e in e
13 interpS (ASSIGN id exp) e = update e id (interpE exp e)
```

Abstract Syntax Tree

Later phases can be achieved through **recursive AST walks**.

How do we tie **errors** in later phases to the source code?

- ▶ In a one-pass compiler, **errors** can be reported using the position information provided by the lexer.
- ▶ In a compiler relying on intermediate languages, such as an AST, the position information provided by the lexer must be stored, e.g., in the AST itself.
- ▶ This allows the type-checker for example to produce useful error messages.

Symbol Table

The **semantic analysis** phase

- ▶ connects variable definitions to their uses
- ▶ checks that expressions are well-typed

Some of that information is maintained in a **symbol table**, which maps identifiers to their types and locations.

Operations:

- ▶ identifiers are **inserted** in the symbol table when declared
- ▶ they are **looked up** in the symbol table when used

Symbol Table

An identifier has a **scope**, which is the part of the program where that identifier is visible, i.e., can be referred to.

Declaring a variable may hide another variable with the same name.

For example (in Haskell):

```
1 x :: Int
2 x = 3
3
4 y :: Bool
5 y = x where
6   x = True
```

- ▶ The scope of `x` starts line 2.
- ▶ The second `x` hides the first in `y`'s body.

Symbol Table

Symbol tables can be implemented in

- ▶ **an imperative style**
- ▶ a functional style (omitted here)

Imperative style:

- ▶ a hash table, where i th bucket is a linked list for all the elements whose keys hash to i
- ▶ linked list might contain types to be used for type-checking
- ▶ to **insert** a declared int x that hashes to i : if the current table maps i to L , insert (x, int) at the front of L
- ▶ if x was already in L , it will now be hidden by the new entry
- ▶ to **lookup** an identifier x that hashes to i : if the current table maps i to L , search for the first entry in L for x
- ▶ to **pop** an identifier x that hashes to i : if the current table maps i to L , remove the first element in L , which must be for x

Symbol Table

When do we pop an identifier?

Assume that an assignment of the form $id := E$ declares id to be an integer, and initializes it with E

For example, in

$$[x := 42; y := 17]$$

how do we know that we have to pop y when closing the block?

We rely on an **auxiliary stack**:

- ▶ we **push** a special marker when **entering** a new block
- ▶ identifiers are **pushed** onto the stack when **declared**
- ▶ symbols are **popped** off down to the topmost marker when **leaving** a block

Symbol Table

For example, consider the statement $S: [x := 42; y := x]; z := x$

Assume that x hashes to 0, y to 1, z to 2. Let $\$$ be the initial stack, and $\oplus$ be the special “new block” marker.

Analyze S using a symbol table and an auxiliary stack for scoping

AST	stack	symbol table	actions						
$[x := 42; y := x]; z := x$	\$	<table><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	1	2				
0	1	2							
$[x := 42; y := x]; z := x$	\$	<table><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	1	2				
0	1	2							
$[x := 42; y := x]; z := x$	$\$ \oplus$	<table><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	1	2	push $\oplus$			
0	1	2							
$[x := 42; y := x]; z := x$	$\$ \oplus$	<table><tr><td></td><td></td><td></td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>				0	1	2	
0	1	2							
$[x := 42; y := x]; z := x$	$\$ \oplus x$	<table><tr><td>(x, int)</td><td></td><td></td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	(x, int)			0	1	2	push x ; insert x
(x, int)									
0	1	2							
$[x := 42; y := x]; z := x$	$\$ \oplus x$	<table><tr><td>(x, int)</td><td></td><td></td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	(x, int)			0	1	2	
(x, int)									
0	1	2							
$[x := 42; y := x]; z := x$	$\$ \oplus xy$	<table><tr><td>(x, int)</td><td>(y, int)</td><td></td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>	(x, int)	(y, int)		0	1	2	lookup x ; push y ; insert y
(x, int)	(y, int)								
0	1	2							
$[x := 42; y := x]; z := x$	\$	<table><tr><td></td><td></td><td></td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>				0	1	2	pop y ; pop x
0	1	2							
$[x := 42; y := x]z := x$	\$	<table><tr><td></td><td></td><td></td></tr><tr><td>0</td><td>1</td><td>2</td></tr></table>				0	1	2	failed to lookup x
0	1	2							

Type Checking

A similar approach can be used to **type check** programs, for example, using algorithms such as the ones introduced in Part 1.

Type checking is used to

- ▶ **catch semantic errors**
- ▶ determine the number of **storage units** needed for the variables of the program

Those storage units are used to compute at compile time, relative address where variables will be stored.

Types and relative addresses are stored in the symbol table.

Conclusion

What did we cover today?

- ▶ Semantic actions
- ▶ Abstract syntax
- ▶ Symbol tables

Next time?

- ▶ Intermediate representations